

Design and Analysis of Algorithms (DAA)

Lab-6 Assignment Report

STUDENT NAME
K CHETAN REDDY

REGISTER NUMBER
AP25110071277

GitHub Repository Link:

<https://github.com/kathichetanreddy8106-ui/DESIGN-AND-ANALYSIS-OF-ALGORITHMS>

1. Finding Maximum and Minimum Element using Divide and Conquer

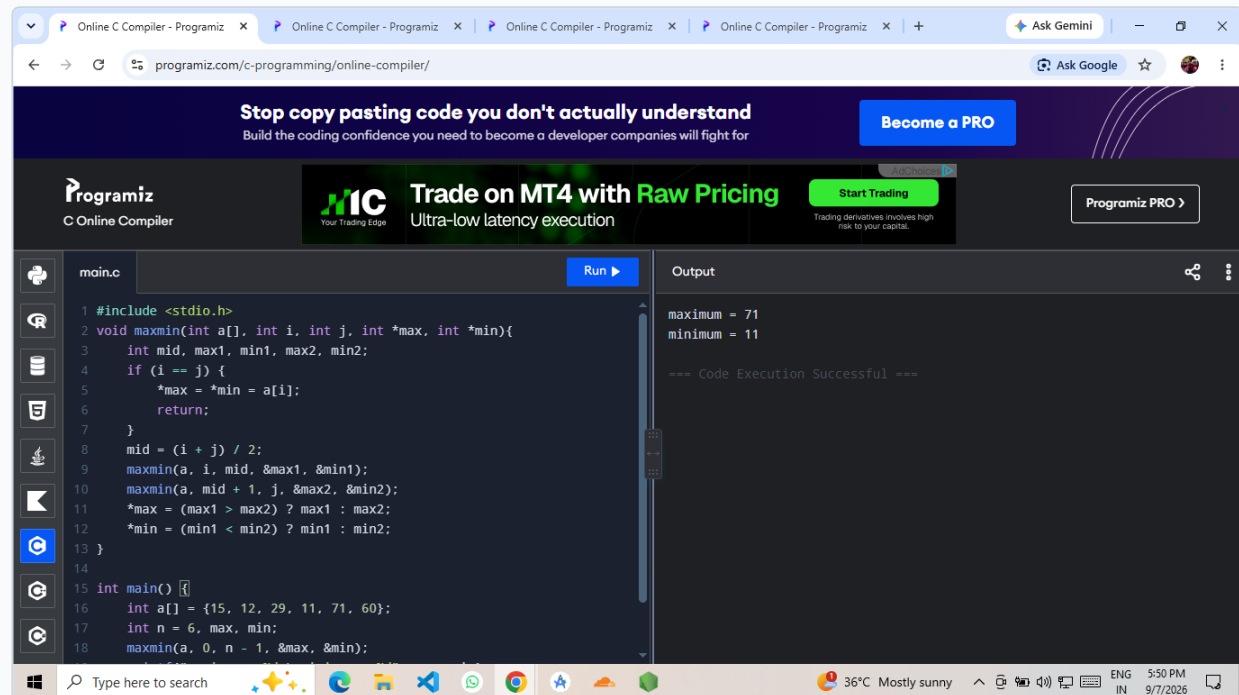
```
#include <stdio.h>
void maxmin(int a[], int i, int j, int *max, int *min){
    int mid, max1, min1, max2, min2;
    if (i == j) {
        *max = *min = a[i];
        return;
    }
    mid = (i + j) / 2;
    maxmin(a, i, mid, &max1, &min1);
    maxmin(a, mid + 1, j, &max2, &min2);
    *max = (max1 > max2) ? max1 : max2;
    *min = (min1 < min2) ? min1 : min2;
}

int main() {
    int a[] = {15, 12, 29, 11, 71, 60};
    int n = 6, max, min;
    maxmin(a, 0, n - 1, &max, &min);
    printf("maximum = %d \nminimum = %d", max, min);
    return 0;
}
```

PROGRAM OUTPUT

```
maximum = 71
minimum = 11
```

COMPILER EXECUTION SCREENSHOT



2. Maximum Subarray using Divide and Conquer

```
#include <stdio.h>
int max(int a, int b) {
    return (a > b) ? a : b;
}
int maxsubarray(int a[], int l, int h) {
    if (l == h)
        return a[l];
    int mid = (l + h) / 2;
    int le, r, sum, lsum = -9999, rsum = -9999, i;
    le = maxsubarray(a, l, mid);
    r = maxsubarray(a, mid + 1, h);
    sum = 0;
    for (i = mid; i >= l; i--) {
        sum += a[i];
        lsum = max(lsum, sum);
    }
    sum = 0;
    for (i = mid + 1; i <= h; i++) {
        sum += a[i];
        rsum = max(rsum, sum);
    }
    return max(max(le, r), lsum + rsum);
}
```

```

}

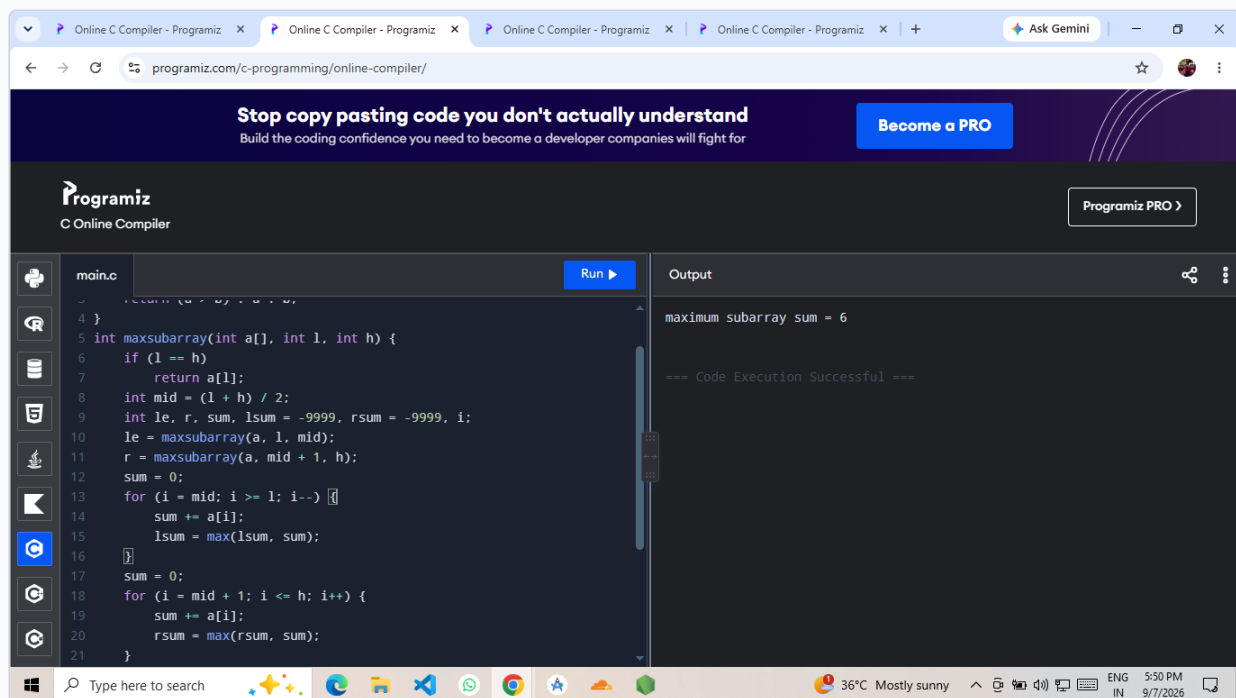
int main() {
    int a[] = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
    int n = 9;
    printf("maximum subarray sum = %d \n", maxsubarray(a, 0, n - 1));
    return 0;
}

```

PROGRAM OUTPUT

```
maximum subarray sum = 6
```

COMPILER EXECUTION SCREENSHOT



3. Strassen's Matrix Multiplication

```

#include <stdio.h>

void strassen(int A[2][2], int B[2][2], int C[2][2]) {
    int a = A[0][0], b = A[0][1], c = A[1][0], d = A[1][1];
    int e = B[0][0], f = B[0][1], g = B[1][0], h = B[1][1];
    int m1 = (a + d) * (e + h);
    int m2 = (c + d) * e;
    int m3 = a * (f - h);
    int m4 = d * (g - e), m5 = (a + b) * h, m6 = (c - a) * (e + f);

```

```

    int m7 = (b - d) * (g + h);
    C[0][0] = m1 + m4 - m5 + m7;
    C[0][1] = m3 + m5;
    C[1][0] = m2 + m4;
    C[1][1] = m1 - m2 + m3 + m6;
}

int main() {
    int A[2][2] = {{1, 2}, {3, 4}};
    int B[2][2] = {{5, 6}, {7, 8}};
    int C[2][2] = {0, 0};
    int i, j;
    strassen(A, B, C);
    printf("result :\n");
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 2; j++) {
            printf("%d ", C[i][j]);
        }
        printf("\n");
    }
    return 0;
}

```

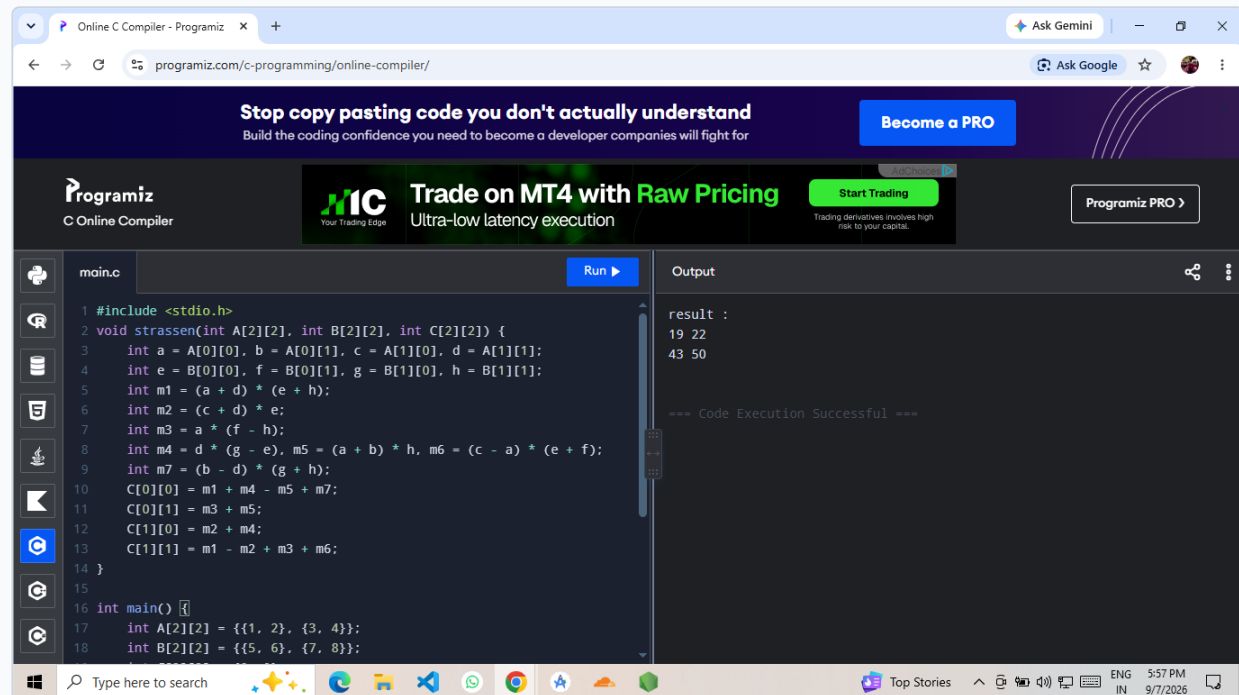
PROGRAM OUTPUT

```

result :
19 22
43 50

```

COMPILER EXECUTION SCREENSHOT



4. Fractional Knapsack using Greedy Approach

```
#include <stdio.h>
struct item {
    int w, v;
};

void knapsack(struct item a[], int n, int c) {
    float total = 0;
    int i, j;
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            float r1 = (float)a[i].v / a[i].w;
            float r2 = (float)a[j].v / a[j].w;
            if (r1 < r2) {
                struct item temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
    for (i = 0; i < n && c > 0; i++) {
        if (a[i].w <= c) {
            c -= a[i].w;
```

```

        total += a[i].v;
    } else {
        total += (float)a[i].v / a[i].w * c;
        c = 0;
    }
}
printf("maximum profit = %.2f\n", total);
}

int main() {
    struct item a[] = {{10, 60}, {20, 100}, {30, 120}};
    int n = 3, c = 50;
    knapsack(a, n, c);
    return 0;
}

```

PROGRAM OUTPUT

```
maximum profit = 240.00
```

COMPILER EXECUTION SCREENSHOT

